

[14]

✓ Os

```
import os
```

```
print(os.listdir('/content'))
```

▼

```
['.config', 'student-mat.csv', '.student.zip_old', 'student.zip', 'sample_data']
```

[15]

✓ Os

```
import os
```

```
print(os.listdir('/content'))
```

▼

```
['.config', 'student-mat.csv', '.student.zip_old', 'student.zip', 'sample_data']
```

[16]

✓ Os

```
import pandas as pd
```

```
df_math = pd.read_csv(  
    '/content/student-mat.csv',  
    sep=';' )
```

```
print("Mathematics dataset loaded successfully!")  
print("Rows:", df_math.shape[0])  
print("Columns:", df_math.shape[1])
```

▼

```
Mathematics dataset loaded successfully!  
Rows: 395  
Columns: 33
```



Files

bin boot content datalab dev etc home kaggle lib lib32 lib64 libx32 media mnt

Disk 87.76 GB available

[1]
✓ 3s

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')
```

[13]

```
import os

print(os.listdir('/content'))
```



Choose Files student-mat.csv
student-mat.csv(text/csv) - 56993 bytes, last modified: 8/8/2026 - 100% done

[17] ✓ 14s

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files student-por.csv

student-por.csv(text/csv) - 93220 bytes, last modified: 8/8/2026 - 100% done

Saving student-por.csv to student-por.csv

[18] ✓ 0s

```
import pandas as pd
```

```
df_math = pd.read_csv(  
    '/content/student-mat.csv',  
    sep=';' )
```

```
df_por = pd.read_csv(  
    '/content/student-por.csv',  
    sep=';' )
```

```
print("Mathematics dataset:", df_math.shape)
```

```
print("Portuguese dataset:", df_por.shape)
```

Mathematics dataset: (395, 33)

Portuguese dataset: (649, 33)

[17] ✓ 14s

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files student-por.csv

student-por.csv(text/csv) - 93220 bytes, last modified: 8/8/2026 - 100% done

Saving student-por.csv to student-por.csv

[18] ✓ 0s

```
import pandas as pd
```

```
df_math = pd.read_csv(  
    '/content/student-mat.csv',  
    sep=';' )
```

```
df_por = pd.read_csv(  
    '/content/student-por.csv',  
    sep=';' )
```

```
print("Mathematics dataset:", df_math.shape)
```

```
print("Portuguese dataset:", df_por.shape)
```

Mathematics dataset: (395, 33)

Portuguese dataset: (649, 33)

▼

...

Mathematics dataset: (395, 33)
Portuguese dataset: (649, 33)

[19]

✓ Os

▼

df_math.head()

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health	absence
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3	1
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5	

5 rows × 33 columns

[20]

✓ Os

▼

df_math.shape

(395, 33)

(\ , \)

[21]

✓ 0s

df_math.columns

```
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',  
      'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',  
      'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',  
      'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc',  
      'Walc', 'health', 'absences', 'G1', 'G2', 'G3'],  
      dtype='object')
```

[22]

✓ 0s

df_math.info()

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 395 entries, 0 to 394  
Data columns (total 33 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   school      395 non-null   object  
1   sex         395 non-null   object  
2   age         395 non-null   int64  
3   address     395 non-null   object  
4   famsize     395 non-null   object  
5   Pstatus     395 non-null   object  
6   Medu        395 non-null   int64  
7   Fedu        395 non-null   int64  
8   Mjob        395 non-null   object  
9   Fjob        395 non-null   object  
10  reason      395 non-null   object  
11  guardian    395 non-null   object  
12  traveltime  395 non-null   int64  
..  ...  
..  ...  
..  ...
```


▶

df_math.describe()

...

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	Walc
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304	3.235443	3.108861	1.481013	2.291139
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659	0.998862	1.113278	0.890741	1.287897
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000	3.000000	2.000000	1.000000	1.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000	3.000000	3.000000	1.000000	2.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000	4.000000	4.000000	2.000000	3.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000	5.000000	5.000000	5.000000	5.000000

[24]

df_math.isnull().sum()

age	0
address	0
famsize	0
Pstatus	0
Medu	0
Fedu	0

dtype: int64

[25]
✓ 0s

```
df_math.duplicated().sum()
```

```
np.int64(0)
```

[26]
✓ 0s

```
df_math.select_dtypes(include='object').columns
```

```
Index(['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob',  
      'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities',  
      'nursery', 'higher', 'internet', 'romantic'],  
      dtype='object')
```

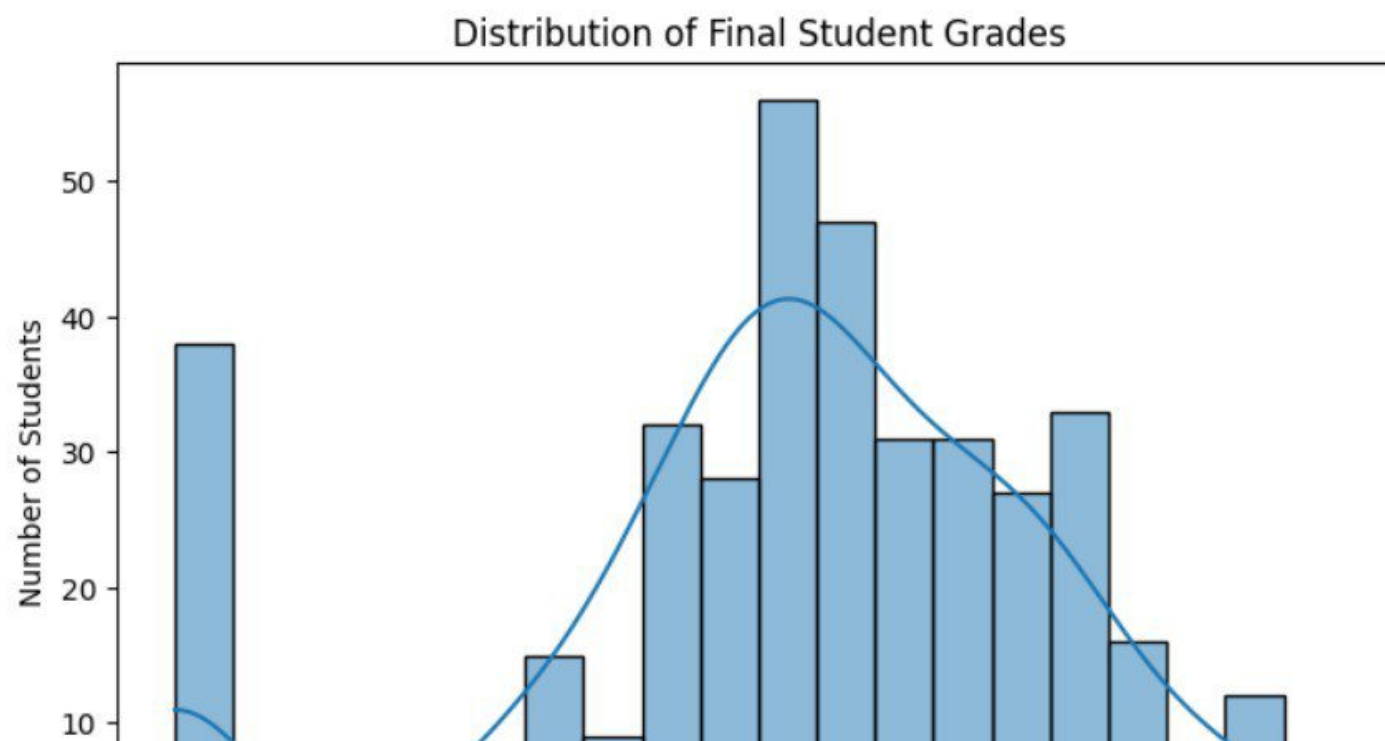

[27]
✓ 1s

```
plt.figure(figsize=(8,5))

sns.histplot(df_math['G3'], bins=20, kde=True)

plt.title('Distribution of Final Student Grades')
plt.xlabel('Final Grade (G3)')
plt.ylabel('Number of Students')

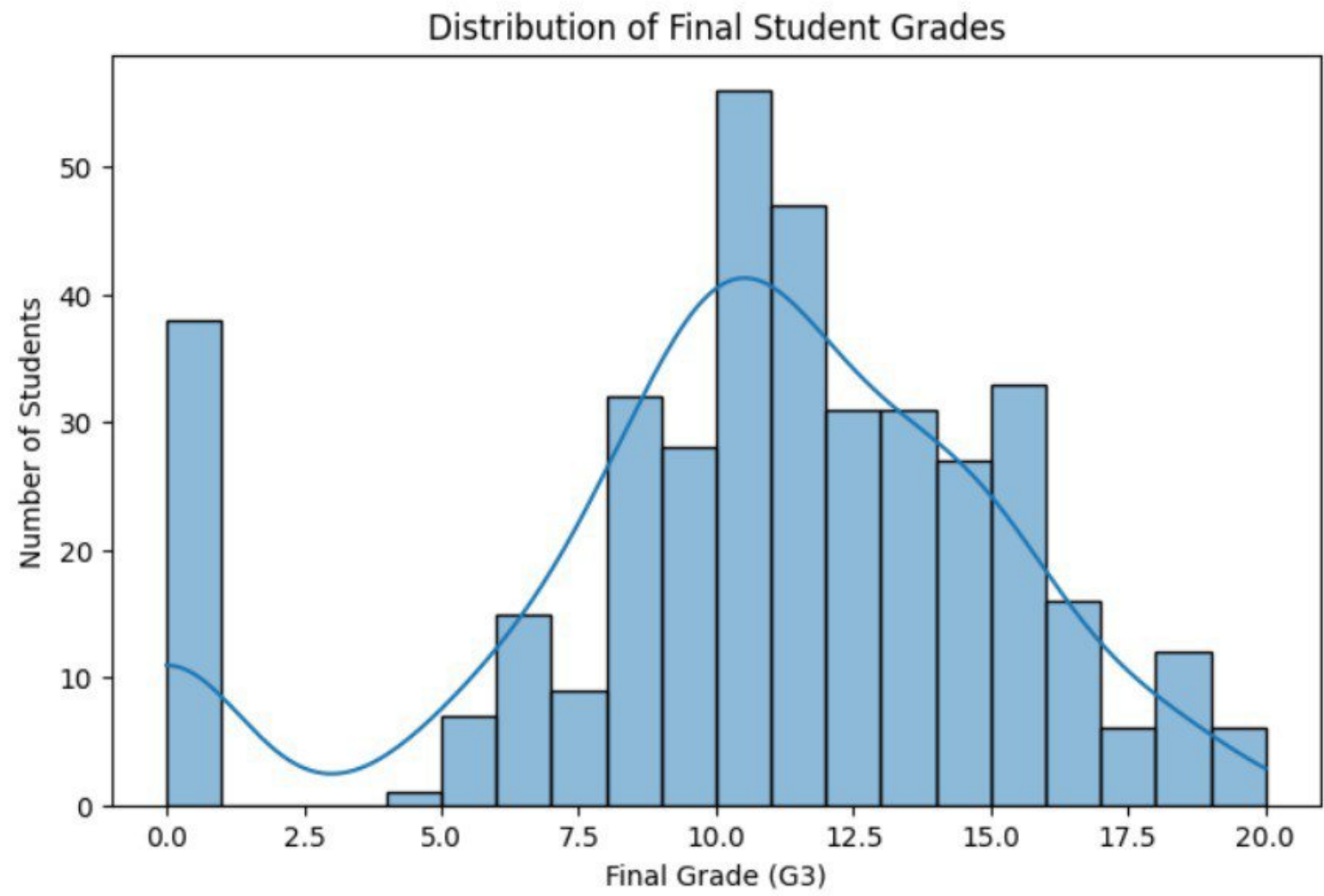
plt.show()
```



✓ 1s

▼

plt.show()



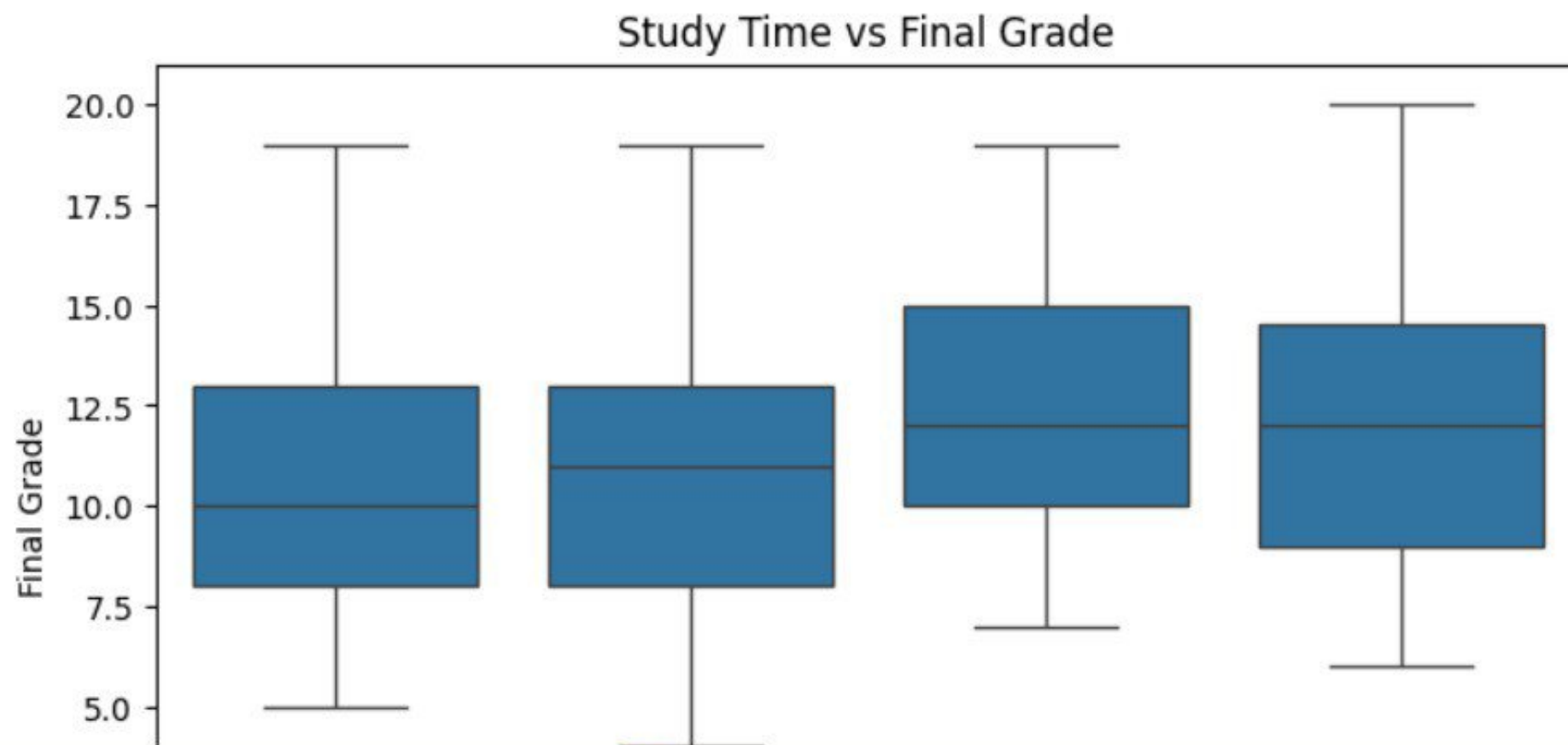
```
plt.figure(figsize=(8,5))

sns.boxplot(x='studytime', y='G3', data=df_math)

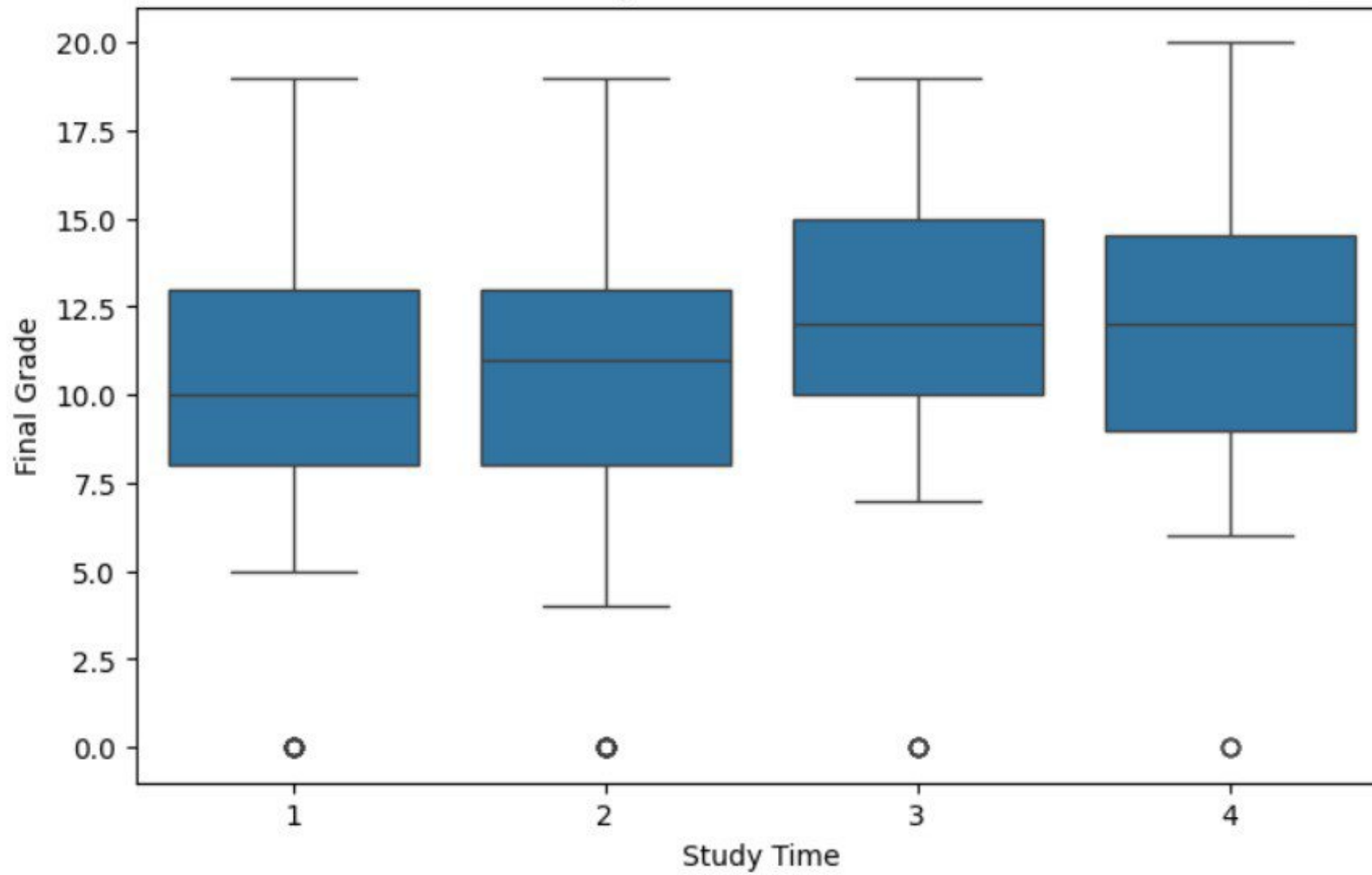
plt.title('Study Time vs Final Grade')
plt.xlabel('Study Time')
plt.ylabel('Final Grade')

plt.show()
```

...



Study Time vs Final Grade

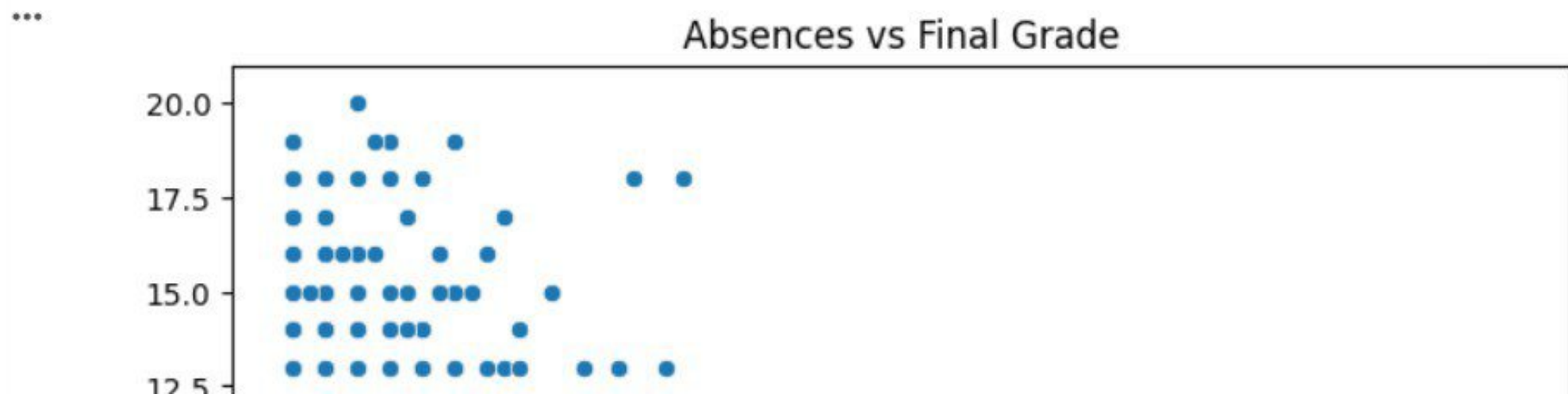


```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='absences',
    y='G3',
    data=df_math
)

plt.title('Absences vs Final Grade')
plt.xlabel('Number of Absences')
plt.ylabel('Final Grade')

plt.show()
```



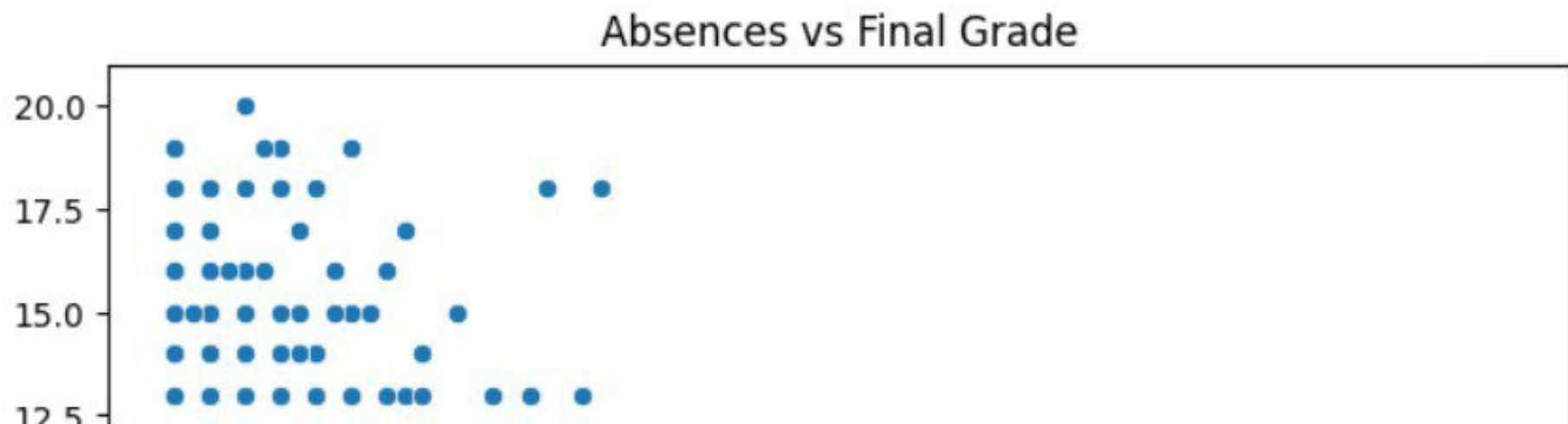

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='absences',
    y='G3',
    data=df_math
)

plt.title('Absences vs Final Grade')
plt.xlabel('Number of Absences')
plt.ylabel('Final Grade')

plt.show()
```

...

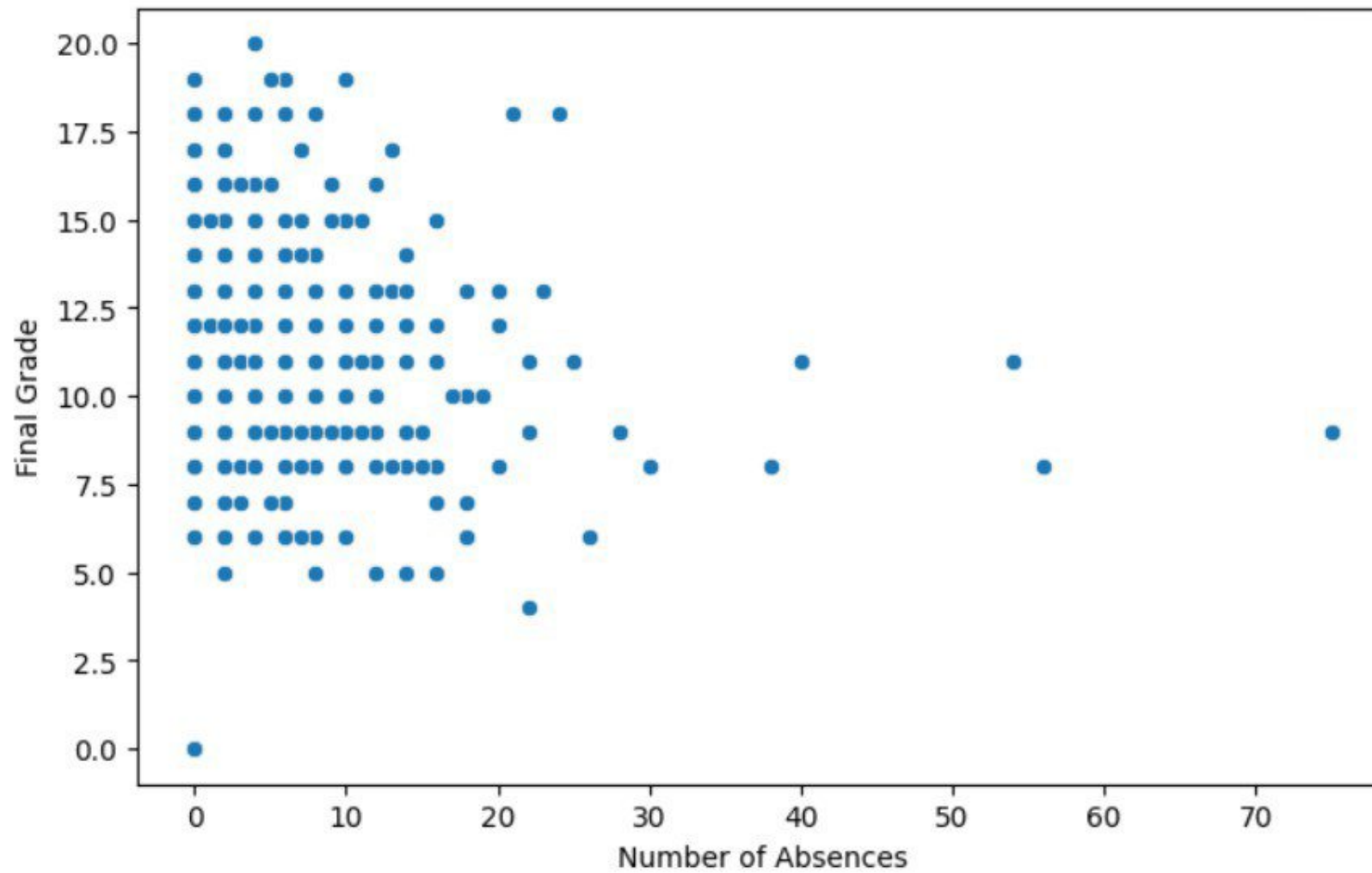




plt.show()

...

Absences vs Final Grade



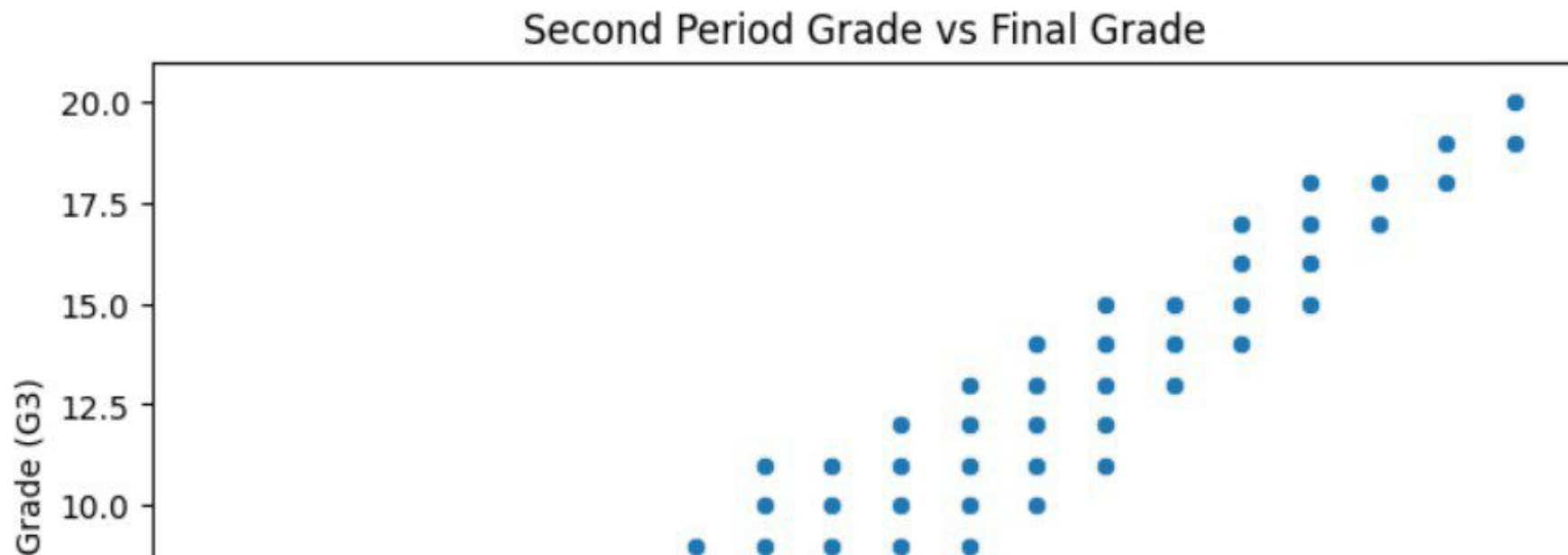
```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='G2',
    y='G3',
    data=df_math
)

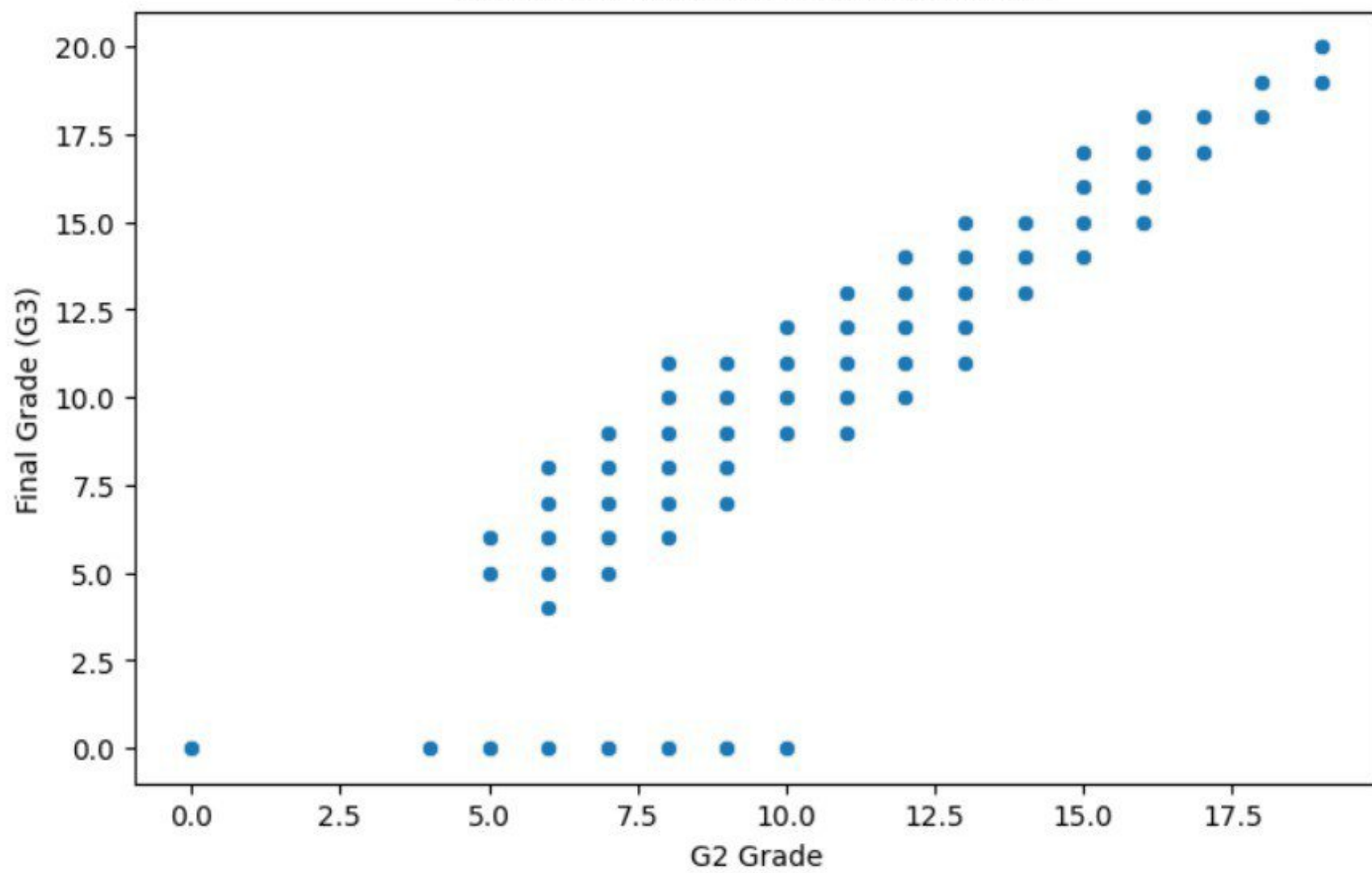
plt.title('Second Period Grade vs Final Grade')
plt.xlabel('G2 Grade')
plt.ylabel('Final Grade (G3)')

plt.show()
```

...



Second Period Grade vs Final Grade



```
▶ plt.figure(figsize=(8,6))

plt.scatter(
    y_test,
    y_pred_tree
)

plt.xlabel('Actual Final Grade')
plt.ylabel('Predicted Final Grade')

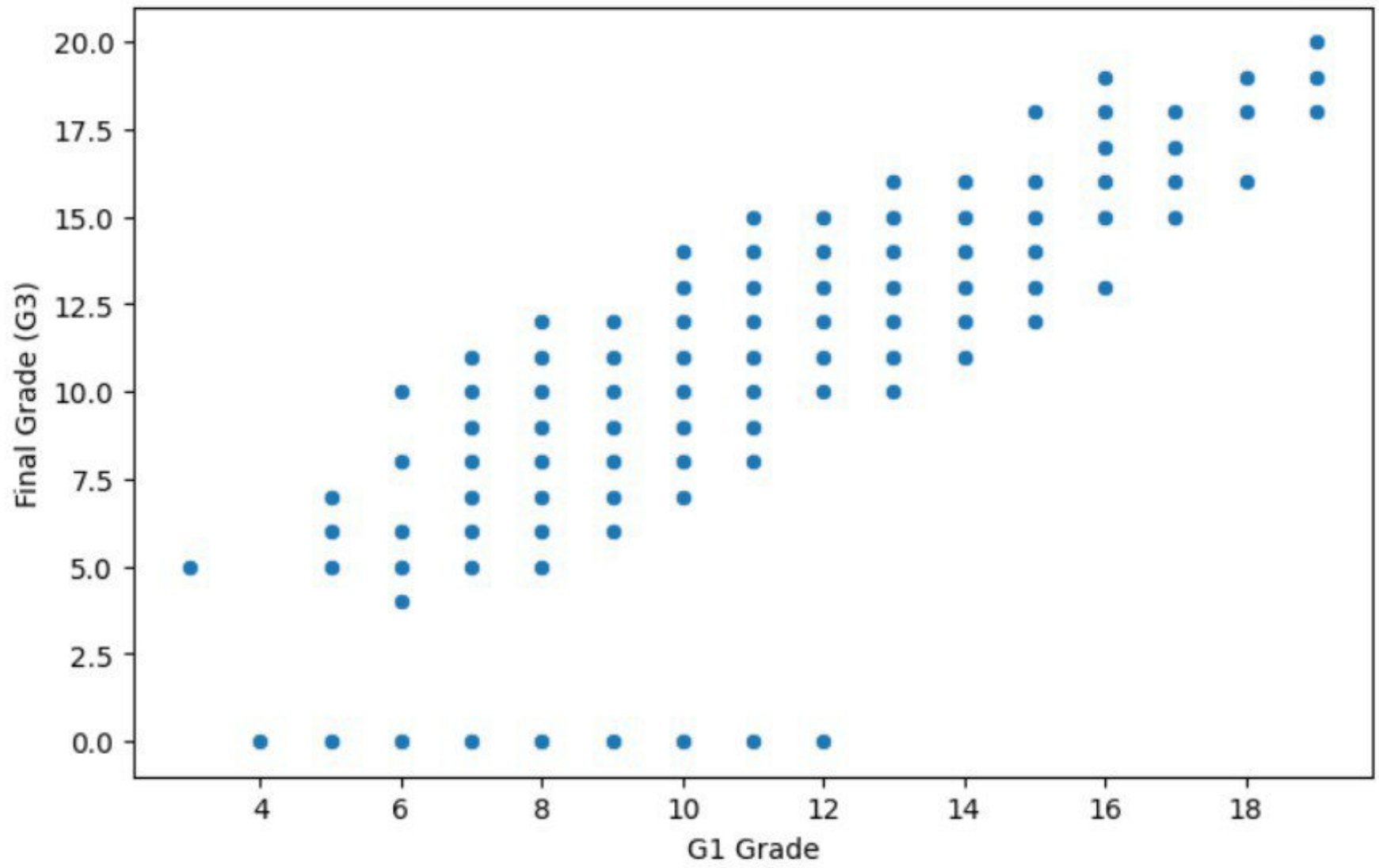
plt.title('Actual vs Predicted Grades - Decision Tree')

plt.show()
```

plt.show()

...

First Period Grade vs Final Grade



```

plt.figure(figsize=(12,8))

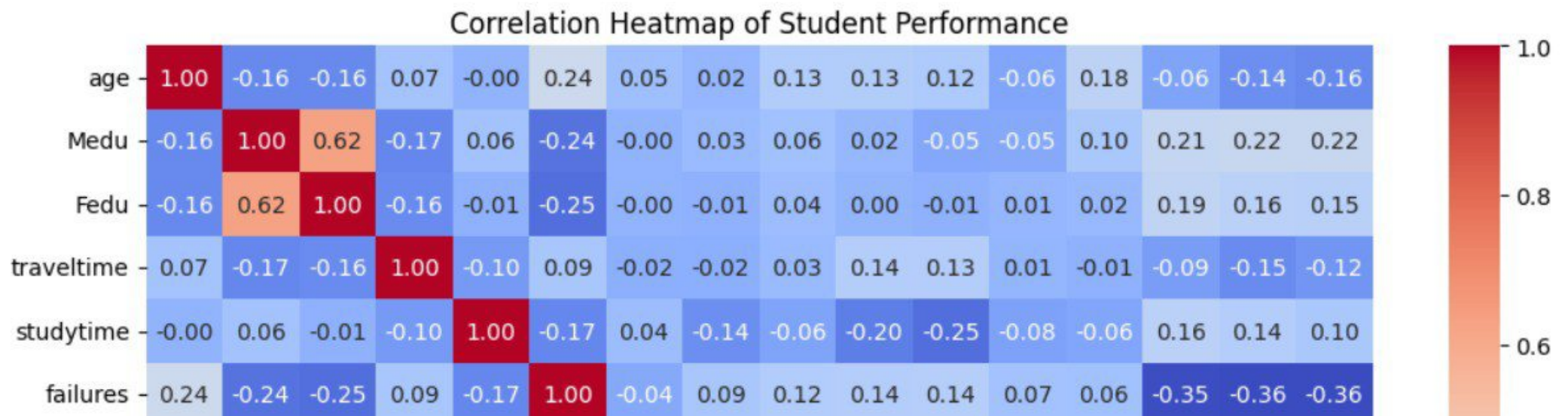
sns.heatmap(
    df_math.select_dtypes(include=np.number).corr(),
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation Heatmap of Student Performance')

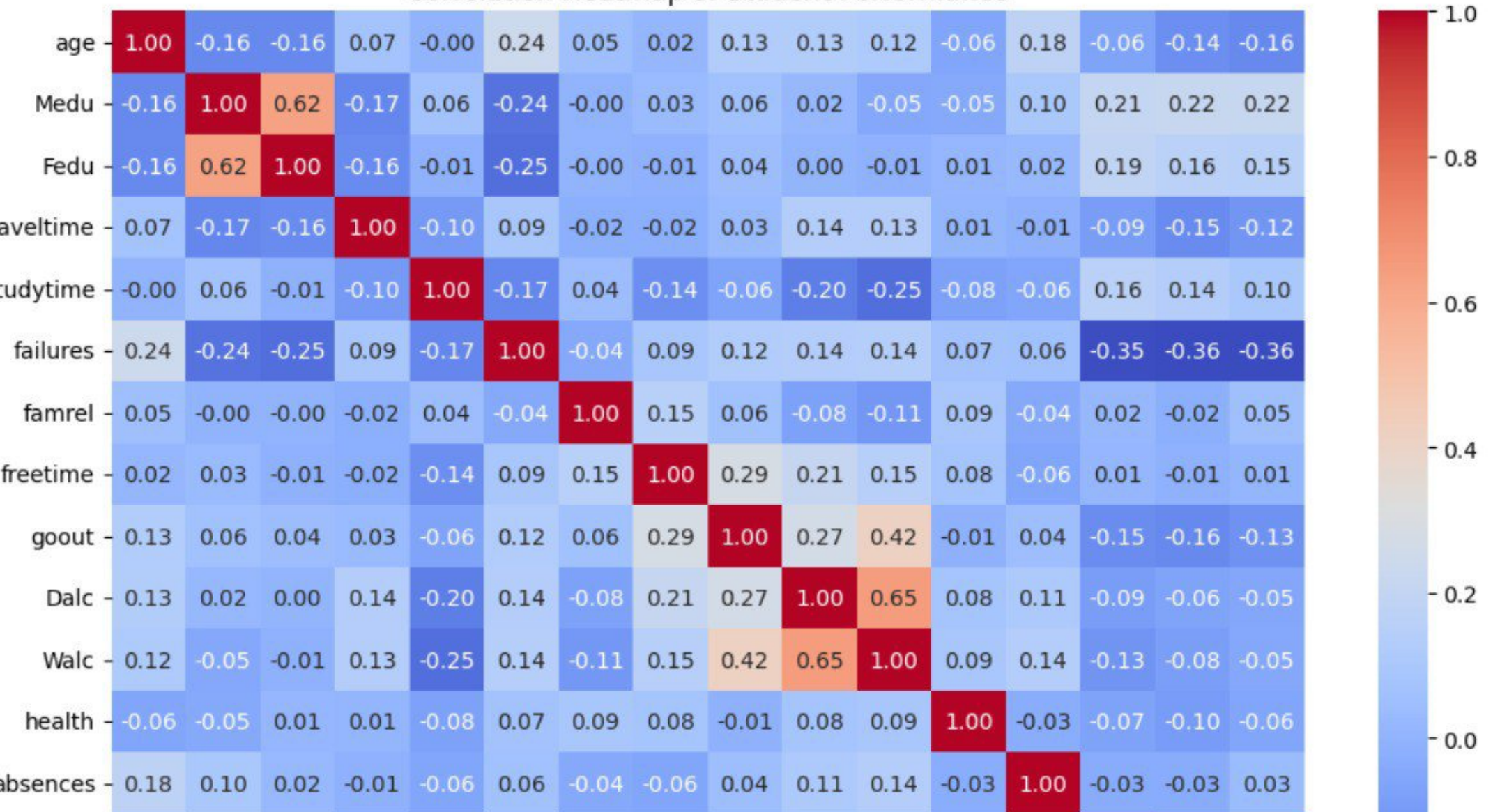
plt.show()

```

...



Correlation Heatmap of Student Performance



[33]
✓ Os

```
y = df_math['G3']
```

[34]
✓ Os

```
X = df_math.drop('G3', axis=1)
```

[35]
✓ Os

```
▶ categorical_features = X.select_dtypes(include=['object']).columns
```

```
numerical_features = X.select_dtypes(  
    include=['int64', 'float64']  
) .columns
```

```
print("Categorical Features:")  
print(categorical_features)
```

```
print("\nNumerical Features:")  
print(numerical_features)
```

✓

```
... Categorical Features:  
Index(['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob',  
      'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities',  
      'nursery', 'higher', 'internet', 'romantic'],  
      dtype='object')  
  
Numerical Features:  
Index(['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel',  
      'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2'],
```

```
dtype= object )
```

[36]

✓ 0s

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', StandardScaler(), numerical_features),  
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)  
    ]  
)
```

[37]

✓ 0s

```
▶ x_train, x_test, y_train, y_test = train_test_split(  
    x,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

[38]

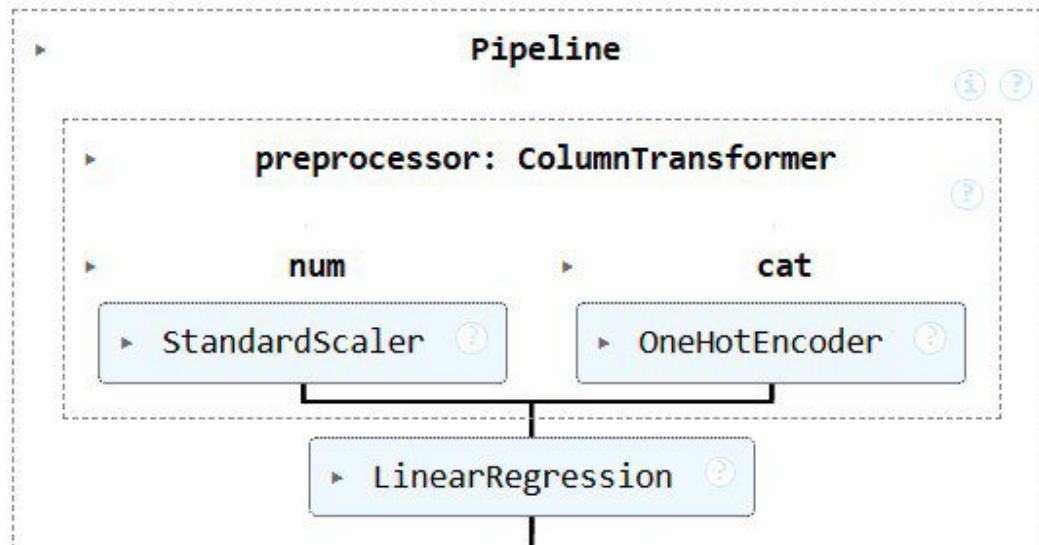
✓ 0s

```
linear_model = Pipeline(  
    steps=[  
        ('preprocessor', preprocessor),  
        ('model', LinearRegression())  
    ]  
)
```

[39]

✓ 4s

```
linear_model.fit(X_train, y_train)
```



[40]

✓ 0s

```
y_pred_linear = linear_model.predict(X_test)
```

[41]

✓ 0s

```
rmse_linear = np.sqrt(  
    mean_squared_error(y_test, y_pred_linear)  
)
```

[42]

✓ 0s

```
r2_linear = r2_score(  
    y_test,  
    y_pred_linear  
)
```


[43]

✓ 0s

```
print("Linear Regression RMSE:", rmse_linear)
print("Linear Regression R²:", r2_linear)
```

▼

```
Linear Regression RMSE: 2.3783697847961367
Linear Regression R²: 0.7241341236974022
```

[44]

✓ 0s

```
decision_tree_model = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', DecisionTreeRegressor(
            random_state=42
        ))
    ]
)
```

[46]

✓ 0s

```
decision_tree_model.fit(
    X_train,
    y_train
)
```

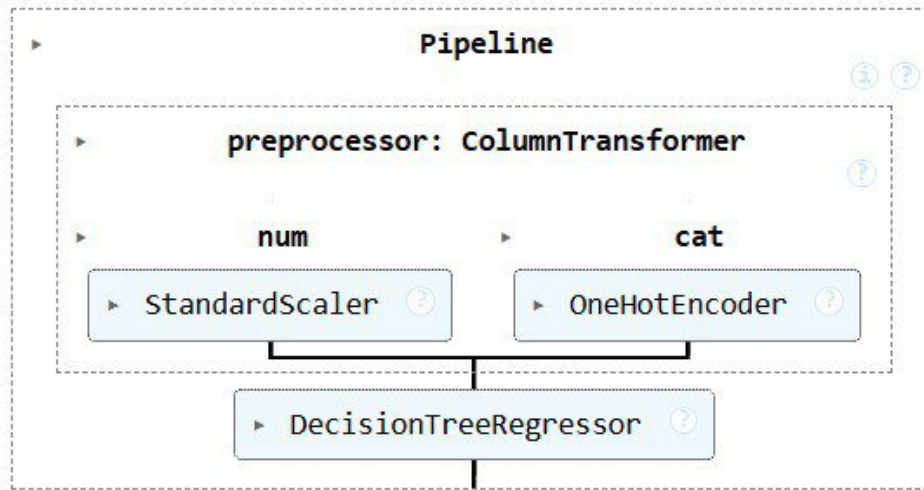
[46]

✓ 0s

```
▶ decision_tree_model.fit(  
    x_train,  
    y_train  
)
```

▼

...



[47]

✓ 0s

```
y_pred_tree = decision_tree_model.predict(x_test)
```

[48]

✓ 0s

```
rmse_tree = np.sqrt(  
    mean_squared_error(y_test, y_pred_tree)  
)  
  
r2_tree = r2_score(  
    y_test,  
    y_pred_tree  
)
```

[48]
✓ 0s

```
r2_tree = r2_score(  
    y_test,  
    y_pred_tree  
)  
  
print("Decision Tree RMSE:", rmse_tree)  
print("Decision Tree R²:", r2_tree)
```

▼

```
Decision Tree RMSE: 2.570524253281587  
Decision Tree R²: 0.6777576344825431
```

[49]
✓ 0s



```
results = pd.DataFrame({  
    'Model': [  
        'Linear Regression',  
        'Decision Tree Regression'  
    ],  
    'RMSE': [  
        rmse_linear,  
        rmse_tree  
    ],  
    'R2 Score': [  
        r2_linear,  
        r2_tree  
    ]  
})  
  
results
```


[49]
✓ 0s

```
    r2_score . [
        r2_linear,
        r2_tree
    ]

results
```

▼

	Model	RMSE	R2 Score	
0	Linear Regression	2.378370	0.724134	
1	Decision Tree Regression	2.570524	0.677758	

Next steps:

[Generate code with results](#)

[New interactive sheet](#)

[51]
✓ 26s

```
from google.colab import sheets
sheet = sheets.InteractiveSheet(df=results)
```

▼

... <https://docs.google.com/spreadsheets/d/16EVpoiAg0LSMz3xtlvUUoK6Id54qKZ8mKNdhAYrTjVA/edit#gid=0>

File Edit View Insert Format Data Tools Extensions Help

[52]

✓ 1s

```
▶ plt.figure(figsize=(8,6))

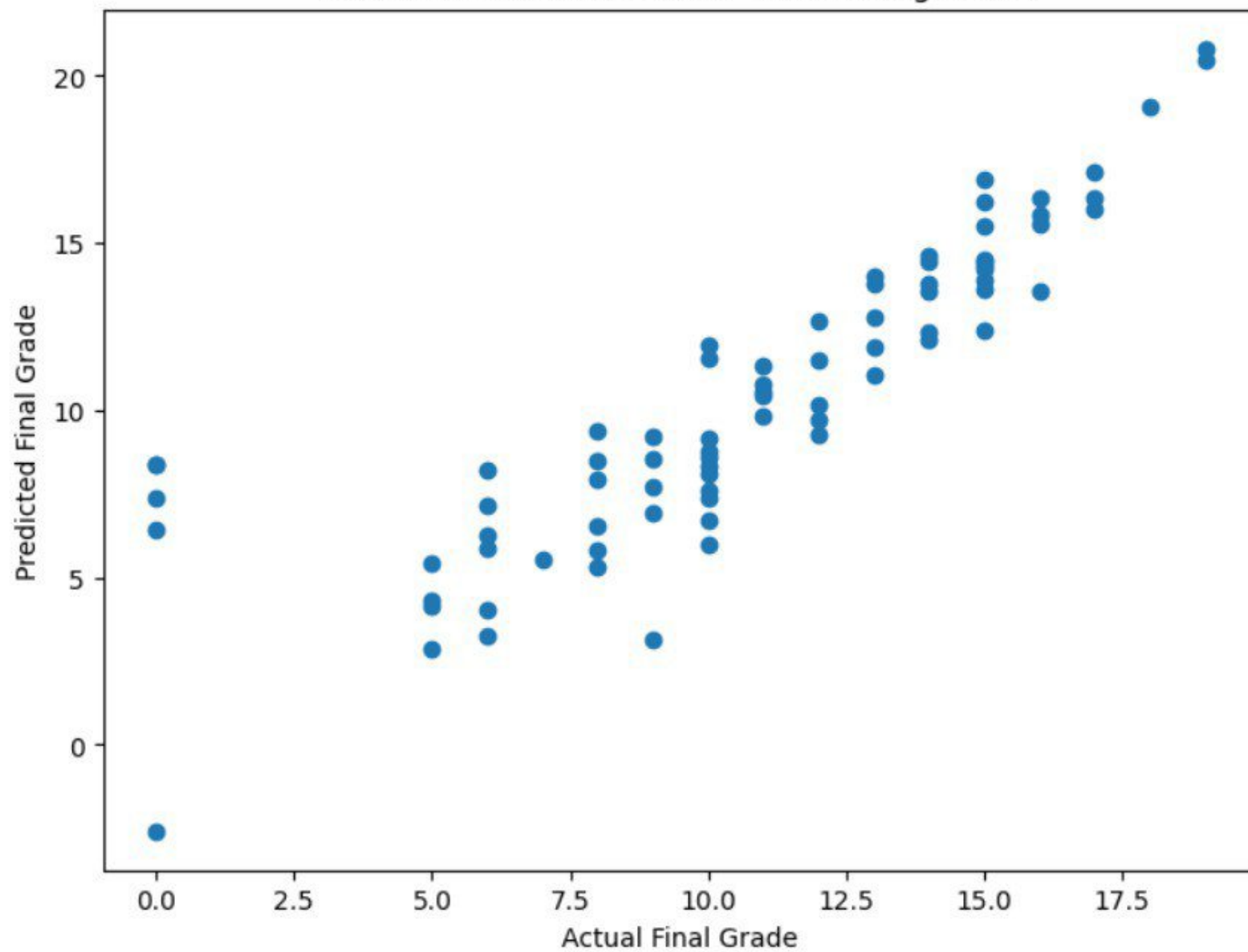
plt.scatter(
    y_test,
    y_pred_linear
)

plt.xlabel('Actual Final Grade')
plt.ylabel('Predicted Final Grade')

plt.title('Actual vs Predicted Grades - Linear Regression')

plt.show()
```

Actual vs Predicted Grades - Linear Regression



```
▶ plt.figure(figsize=(8,6))

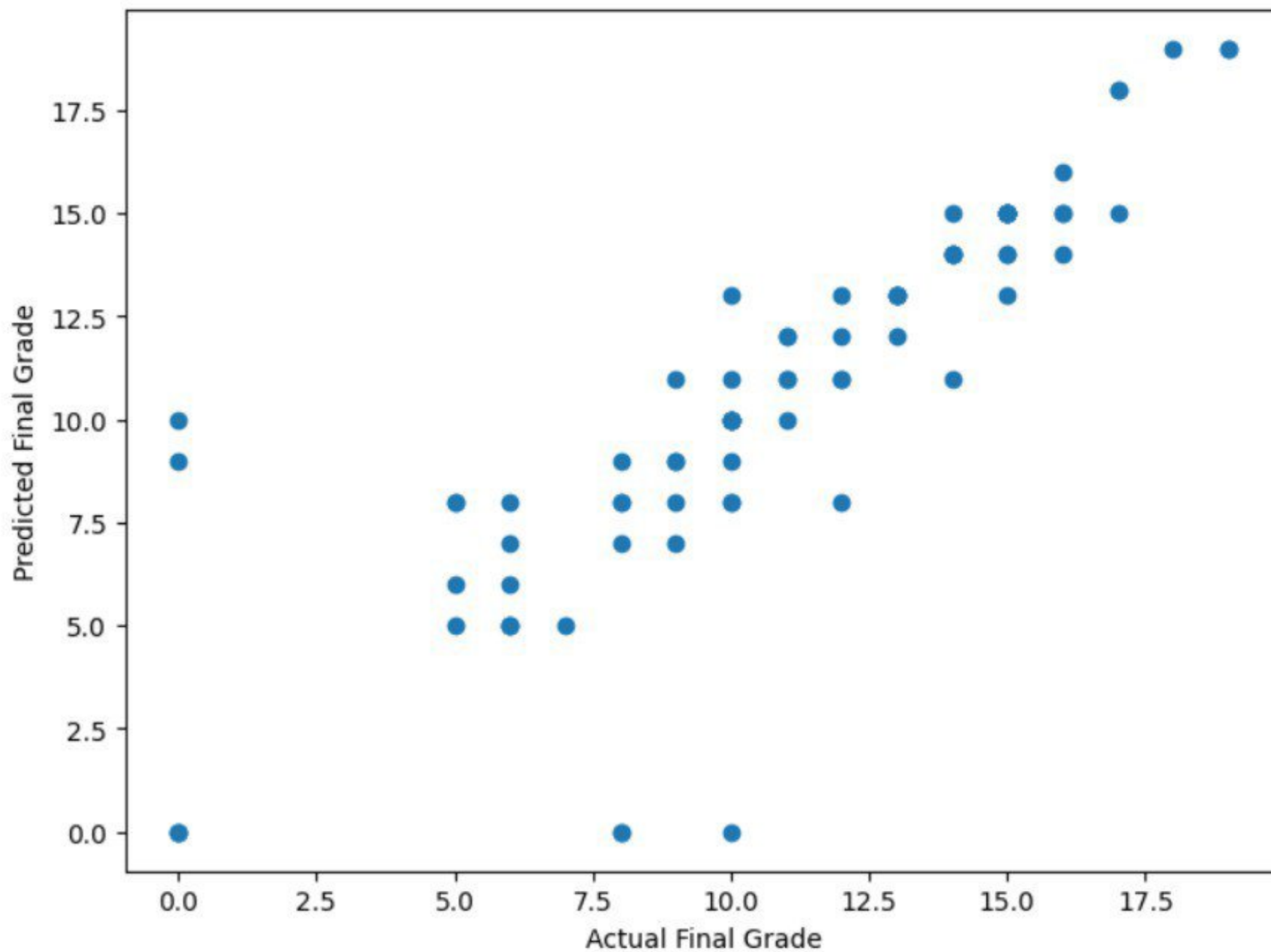
plt.scatter(
    y_test,
    y_pred_tree
)

plt.xlabel('Actual Final Grade')
plt.ylabel('Predicted Final Grade')

plt.title('Actual vs Predicted Grades - Decision Tree')

plt.show()
```

Actual vs Predicted Grades - Decision Tree




```

▶ comparison = pd.DataFrame({
    'Actual G3': y_test.values,
    'Linear Regression Prediction': y_pred_linear,
    'Decision Tree Prediction': y_pred_tree
})

comparison.head(10)

```

...

	Actual G3	Linear Regression Prediction	Decision Tree Prediction
0	10	6.001607	8.0
1	12	11.528478	12.0
2	5	2.866437	8.0
3	10	8.796631	10.0
4	9	8.553106	9.0
5	13	11.889641	13.0
6	18	19.096742	19.0
7	6	7.173079	5.0
8	0	7.405071	9.0
9	14	12.311537	14.0



[65]
✓ 2s

```
from google.colab import sheets
sheet = sheets.InteractiveSheet(df=comparison)
```

<https://docs.google.com/spreadsheets/d/1ZHq7e3r1FohsKnnIwMZqgWP7-7EPcdU6Jqxn7dQuq38/edit#gid=0>

File Edit View Insert Format Data Tools Extensions Help									
100% £ % .0 .00 123 Defaul... - 10 + B I A									
A1	Actual G3								
	A	B	C	D	E	F	G	H	I
1	Actual G3	Linear Regressio	Decision Tree Prediction						
2	10	6.001606988	8						
3	12	11.52847844	12						
4	5	2.866436774	8						
5	10	8.796631165	10						
6	9	8.553106269	9						
7	13	11.88964112	13						
8	18	19.09674195	19						
9	6	7.173079009	5						
10	0	7.405070704	9						
11	14	12.31153657	14						
12	15	14.50202689	15						
13	7	5.524482007	5						
14	15	13.64498491	14						
15	10	11.04404500	12						

```
preprocessor_fitted = decision_tree_model.named_steps['preprocessor']
```

```
feature_names = preprocessor_fitted.get_feature_names_out()
```

```
importance = decision_tree_model.named_steps[  
    'model'  
].feature_importances_
```

```
feature_importance = pd.DataFrame({  
    'Feature': feature_names,  
    'Importance': importance  
})
```

```
feature_importance = feature_importance.sort_values(  
    by='Importance',  
    ascending=False  
)
```

```
feature_importance.head(15)
```

...

	Feature	Importance
--	---------	------------



14	num__G2	0.738286
----	---------	----------

12	num__absences	0.154572
----	---------------	----------

59]
✓ 0s

```
▶ top_features = feature_importance.head(10)

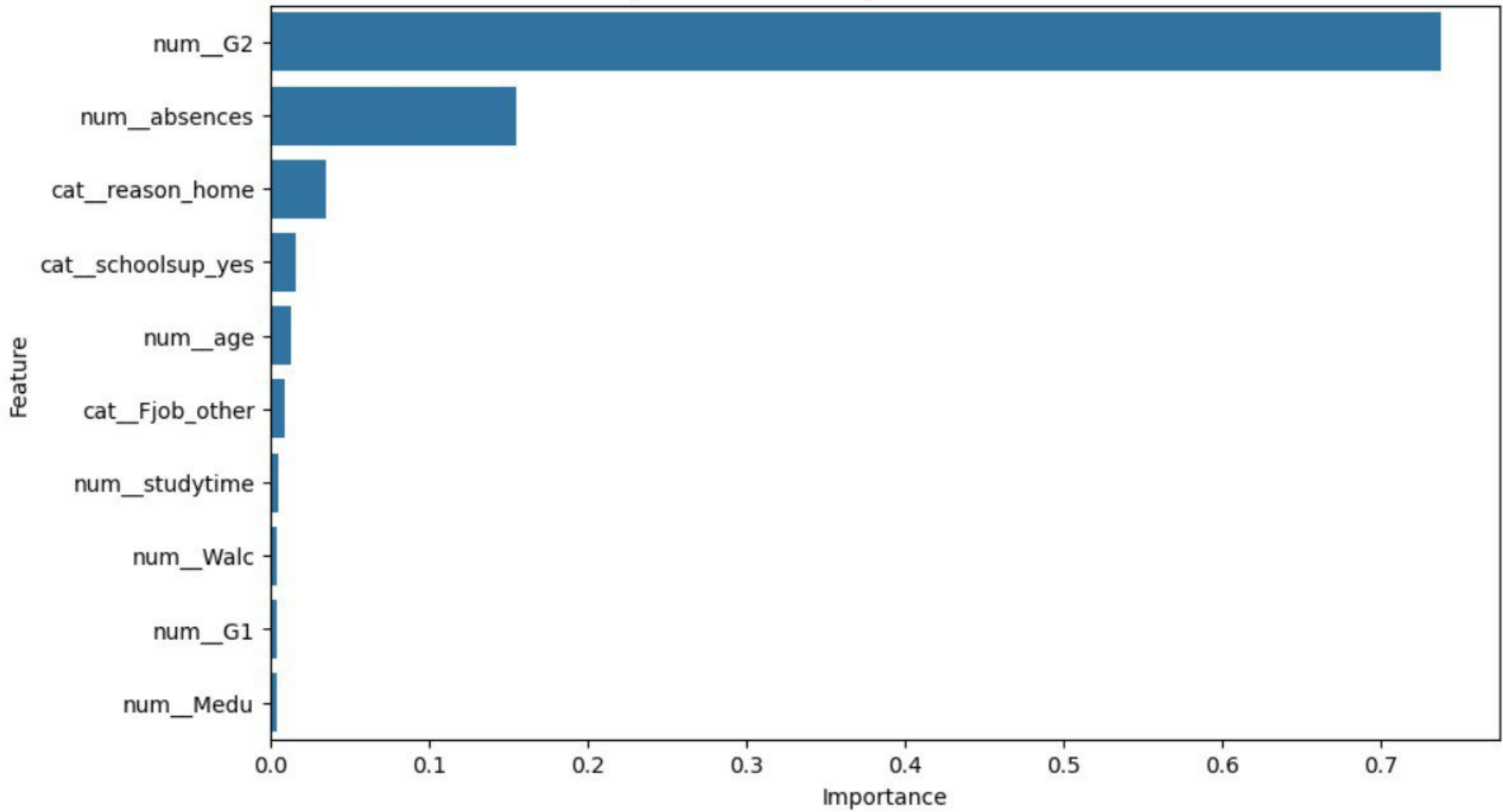
plt.figure(figsize=(10,6))

sns.barplot(
    x='Importance',
    y='Feature',
    data=top_features
)

plt.title('Top Factors Affecting Student Performance')

plt.show()
```

Top Factors Affecting Student Performance




```
df_por = pd.read_csv(
    '/content/student-por.csv',
    sep=';'
)
```

```
df_por.shape
```

```
(649, 33)
```

```
df_por.head()
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health	absence
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5	

5 rows × 33 columns

53]

Os

df_por.info()

✓

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 649 entries, 0 to 648

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
0	school	649 non-null	object
1	sex	649 non-null	object
2	age	649 non-null	int64
3	address	649 non-null	object
4	famsize	649 non-null	object
5	Pstatus	649 non-null	object
6	Medu	649 non-null	int64
7	Fedu	649 non-null	int64
8	Mjob	649 non-null	object
9	Fjob	649 non-null	object
10	reason	649 non-null	object
11	guardian	649 non-null	object
12	traveltime	649 non-null	int64
13	studytime	649 non-null	int64
14	failures	649 non-null	int64
15	schoolsup	649 non-null	object
16	famsup	649 non-null	object
17	paid	649 non-null	object
18	activities	649 non-null	object
19	nursery	649 non-null	object
20	higher	649 non-null	object

dtypes: int64(16), object(17)
memory usage: 167.4+ KB

[64]
✓ 0s

```
def predict_student_performance(model, student_data):  
    ... prediction = model.predict(student_data)  
    ... return prediction[0]
```